

Artificial Intelligence

Logical Agents

Christopher Simpkins

Kennesaw State University

1 Logic and AI

In AI, **knowledge-based** agents use a process of **reasoning** over an internal **representation** of knowledge to decide what actions to take.

2 Knowledge-Based Agents

function KB-AGENT(*percept*) **returns** an *action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

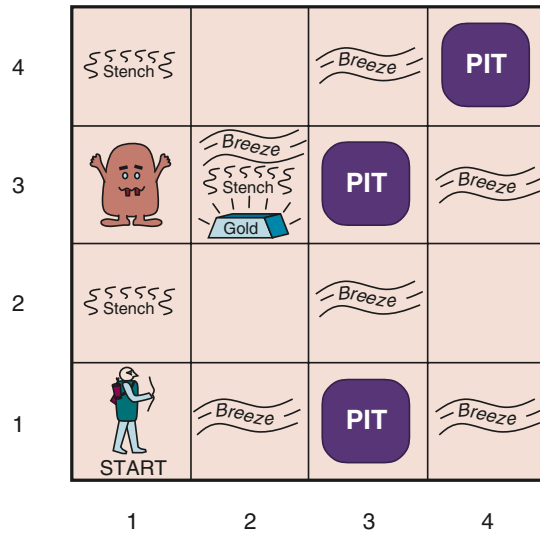
action $\leftarrow$ ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t $\leftarrow$ *t* + 1

return *action*

3 The Wumpus World



4 First Steps Wumpus World

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A	OK		

(a)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
V	A	P?	
OK	B		
	OK		

(b)

5 Later Steps Wumpus World

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

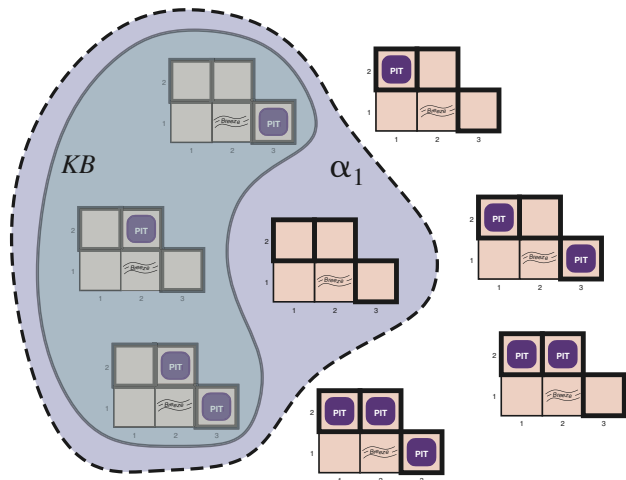
(a)

A = Agent
 B = Breeze
 G = Glitter, Gold
 OK = Safe square
 P = Pit
 S = Stench
 V = Visited
 W = Wumpus

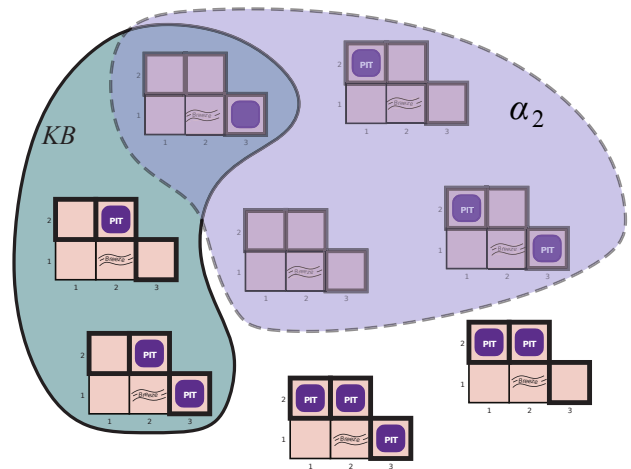
1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(b)

6 Logic

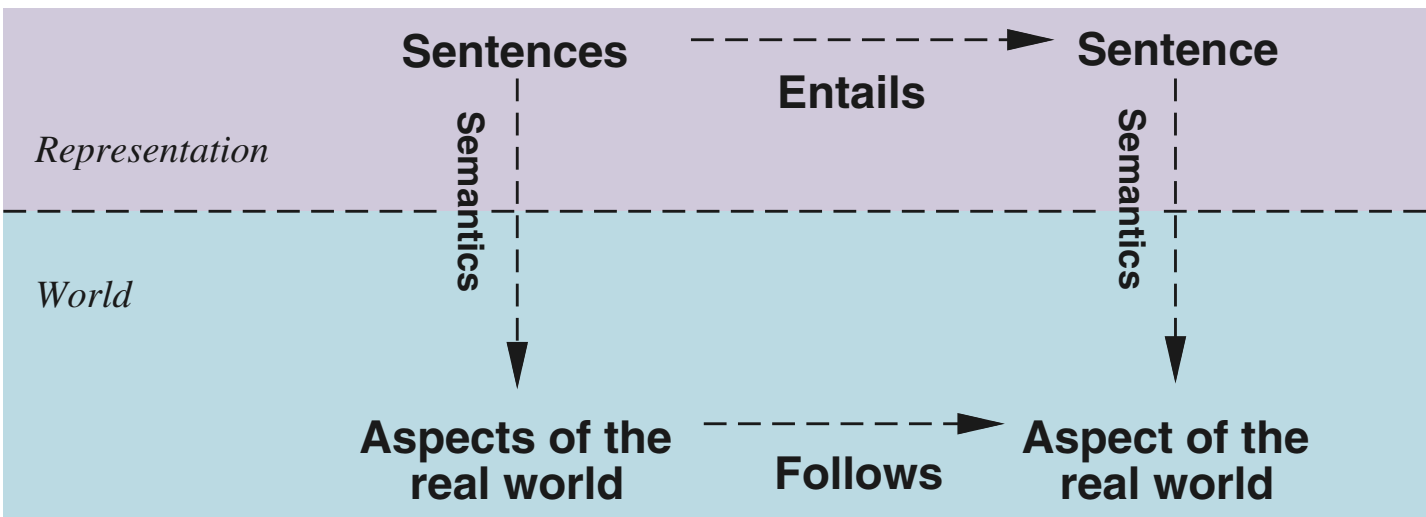


(a)



(b)

7 Logic



8 Propositional Logic

$$\begin{aligned}
 \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
 \textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots \\
 \textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\
 &\mid \neg \textit{Sentence} \\
 &\mid \textit{Sentence} \wedge \textit{Sentence} \\
 &\mid \textit{Sentence} \vee \textit{Sentence} \\
 &\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
 &\mid \textit{Sentence} \Leftrightarrow \textit{Sentence}
 \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

9 Propositional Logic

P	Q	$\neg P$	$P \wedge Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>

10 Propositional Theorem Proving

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>

11 Propositional Theorem Proving

function TT-ENTAILS?(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$symbols \leftarrow$ a list of the proposition symbols in KB and α

return TT-CHECK-ALL($KB, \alpha, symbols, \{ \}$)

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** *true* or *false*

if EMPTY?($symbols$) **then**

if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)

else return true // when KB is false, always return true

else

$P \leftarrow$ FIRST($symbols$)

$rest \leftarrow$ REST($symbols$)

return (TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = true\}$)

and

 TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = false\}$))

12 Propositional Theorem Proving

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$ commutativity of $\wedge$

$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$ commutativity of $\vee$

$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$ associativity of $\wedge$

$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$ associativity of $\vee$

$\neg(\neg\alpha) \equiv \alpha$ double-negation elimination

$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$ contraposition

$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ implication elimination

$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$ biconditional elimination

$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ De Morgan

$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ De Morgan

$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$ distributivity of $\wedge$ over $\vee$

$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$ distributivity of $\vee$ over $\wedge$

13 Propositional Theorem Proving

$$\begin{aligned} \text{CNFSentence} &\rightarrow \text{Clause}_1 \wedge \dots \wedge \text{Clause}_n \\ \text{Clause} &\rightarrow \text{Literal}_1 \vee \dots \vee \text{Literal}_m \\ \text{Fact} &\rightarrow \text{Symbol} \\ \text{Literal} &\rightarrow \text{Symbol} \mid \neg \text{Symbol} \\ \text{Symbol} &\rightarrow P \mid Q \mid R \mid \dots \\ \text{HornClauseForm} &\rightarrow \text{DefiniteClauseForm} \mid \text{GoalClauseForm} \\ \text{DefiniteClauseForm} &\rightarrow \text{Fact} \mid (\text{Symbol}_1 \wedge \dots \wedge \text{Symbol}_l) \Rightarrow \text{Symbol} \\ \text{GoalClauseForm} &\rightarrow (\text{Symbol}_1 \wedge \dots \wedge \text{Symbol}_l) \Rightarrow \text{False} \end{aligned}$$

14 Propositional Theorem Proving

function PL-RESOLUTION(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg \alpha$

$new \leftarrow \{\}$

while *true* **do**

for each pair of clauses C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow \text{PL-RESOLVE}(C_i, C_j)$

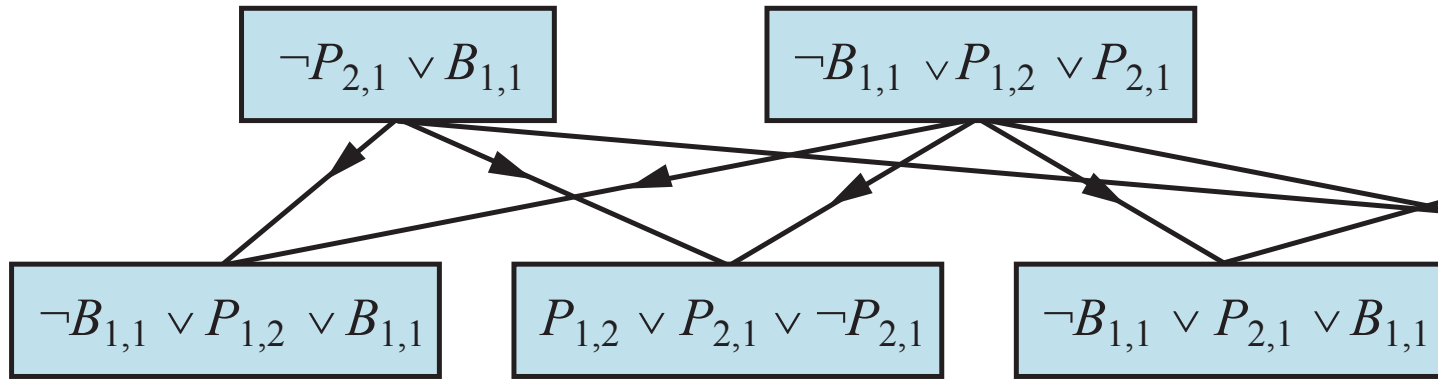
if $resolvents$ contains the empty clause **then return** *true*

$new \leftarrow new \cup resolvents$

if $new \subseteq clauses$ **then return** *false*

$clauses \leftarrow clauses \cup new$

15 Propositional Theorem Proving



16 Propositional Theorem Proving

function PL-FC-ENTAILS?(*KB*, *q*) **returns** *true* or *false*

inputs: *KB*, the knowledge base, a set of propositional definite clauses

q, the query, a proposition symbol

count $\leftarrow$ a table, where *count*[*c*] is initially the number of symbols in clause *c*'s premise

inferred $\leftarrow$ a table, where *inferred*[*s*] is initially *false* for all symbols

queue $\leftarrow$ a queue of symbols, initially symbols known to be true in *KB*

while *queue* is not empty **do**

p $\leftarrow$ POP(*queue*)

if *p* = *q* **then return** *true*

if *inferred*[*p*] = *false* **then**

inferred[*p*] $\leftarrow$ *true*

for each clause *c* in *KB* where *p* is in *c*.PREMISE **do**

 decrement *count*[*c*]

if *count*[*c*] = 0 **then** add *c*.CONCLUSION to *queue*

return *false*

17 Propositional Theorem Proving

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

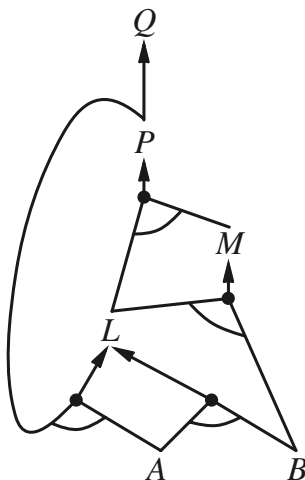
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

(a)



(b)

18 Propositional Model Checking

function DPLL-SATISFIABLE?(s) **returns** *true* or *false*

inputs: s , a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of s

$symbols \leftarrow$ a list of the proposition symbols in s

return DPLL($clauses, symbols, \{\}$)

function DPLL($clauses, symbols, model$) **returns** *true* or *false*

if every clause in $clauses$ is true in $model$ **then return** *true*

if some clause in $clauses$ is false in $model$ **then return** *false*

$P, value \leftarrow$ FIND-PURE-SYMBOL($symbols, clauses, model$)

if P is non-null **then return** DPLL($clauses, symbols - P, model \cup \{P=value\}$)

$P, value \leftarrow$ FIND-UNIT-CLAUSE($clauses, model$)

if P is non-null **then return** DPLL($clauses, symbols - P, model \cup \{P=value\}$)

$P \leftarrow$ FIRST($symbols$); $rest \leftarrow$ REST($symbols$)

return DPLL($clauses, rest, model \cup \{P=true\}$) **or**

DPLL($clauses, rest, model \cup \{P=false\}$)

19 Propositional Model Checking

function WALKSAT(*clauses*, *p*, *max_flips*) **returns** a satisfying model or *failure*

inputs: *clauses*, a set of clauses in propositional logic

p, the probability of choosing to do a “random walk” move, typically around 0.5

max_flips, number of value flips allowed before giving up

model $\leftarrow$ a random assignment of *true/false* to the symbols in *clauses*

for each *i* = 1 **to** *max_flips* **do**

if *model* satisfies *clauses* **then return** *model*

clause $\leftarrow$ a randomly selected clause from *clauses* that is false in *model*

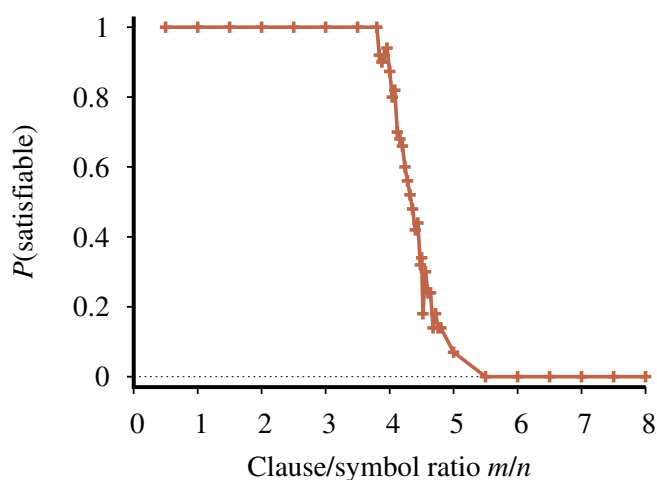
if RANDOM(0, 1) $\leq p$ **then**

 flip the value in *model* of a randomly selected symbol from *clause*

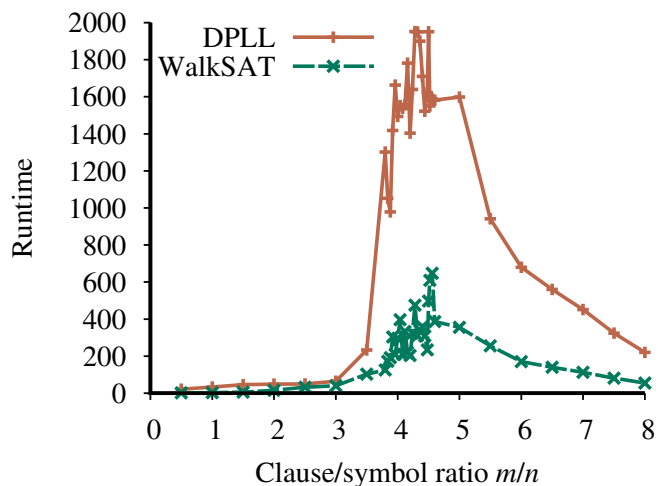
else flip whichever symbol in *clause* maximizes the number of satisfied clauses

return *failure*

20 Propositional Model Checking



(a)



(b)

21 Agents Based on Propositional Logic

function HYBRID-WUMPUS-AGENT(*percept*) **returns** an action
inputs: *percept*, a list, [*stench*, *breeze*, *glitter*, *bump*, *scream*]
persistent: *KB*, a knowledge base, initially the atemporal "wumpus physics"
t, a counter, initially 0, indicating time
plan, an action sequence, initially empty

```

TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
TELL the KB the temporal "physics" sentences for time t
safe ← {[x,y] : ASK(KB,  $OK_{t,y}^i$ ) = true}
if ASK(KB, Glittert) = true then
  plan ← [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
if plan is empty then
  unvisited ← {[x,y] : ASK(KB,  $L'_{t,y}$ ) = false for all  $t' \leq t$ }
  plan ← PLAN-ROUTE(current, unvisited ∩ safe, safe)
if plan is empty and ASK(KB, HaveArrowt) = true then
  possible_wumpus ← {[x,y] : ASK(KB,  $\neg W_{t,y}$ ) = false}
  plan ← PLAN-SHOT(current, possible_wumpus, safe)
if plan is empty then // no choice but to take a risk
  not_unsafe ← {[x,y] : ASK(KB,  $\neg OK_{t,y}^i$ ) = false}
  plan ← PLAN-ROUTE(current, unvisited ∩ not_unsafe, safe)
if plan is empty then
  plan ← PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
action ← POP(plan)
TELL(KB, MAKE-ACTION-SENTENCE(action, t))
t ← t + 1
return action

```

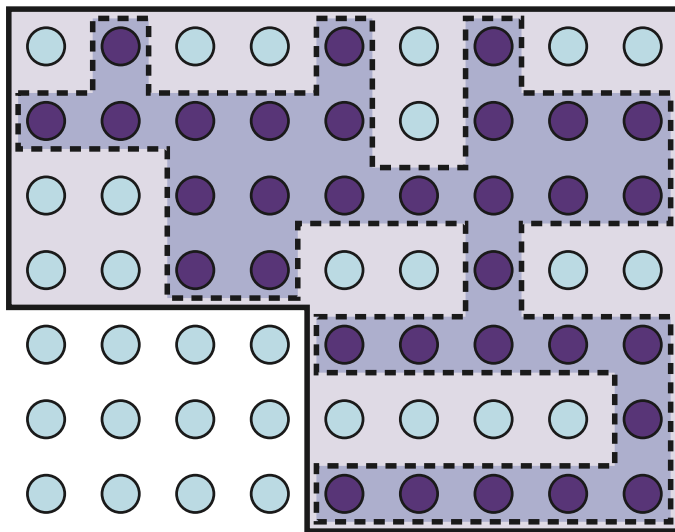
function PLAN-ROUTE(*current*, *goals*, *allowed*) **returns** an action sequence
inputs: *current*, the agent's current position
goals, a set of squares; try to plan a route to one of them
allowed, a set of squares that can form part of the route

```

problem ← ROUTE-PROBLEM(current, goals, allowed)
return SEARCH(problem) // Any search algorithm from Chapter 3

```

22 Agents Based on Propositional Logic



23 Agents Based on Propositional Logic

function SATPLAN(*init*, *transition*, *goal*, $T_{\max}$) **returns** solution or *failure*

inputs: *init*, *transition*, *goal*, constitute a description of the problem

$T_{\max}$, an upper limit for plan length

for $t = 0$ **to** $T_{\max}$ **do**

$cnf \leftarrow \text{TRANSLATE-TO-SAT}(init, transition, goal, t)$

$model \leftarrow \text{SAT-SOLVER}(cnf)$

if *model* is not null **then**

return EXTRACT-SOLUTION(*model*)

return *failure*